

DSA Comprehensive Question Bank

(200 MCQs)

Introduction to DSA

Q1. [Def] What is an algorithm?

- A) A programming language
- B) A step-by-step procedure for solving a problem
- C) A type of hardware
- D) A data structure

Q2. [Def] Space complexity is defined as:

- A) Time taken by a program
- B) Maximum memory consumed by the algorithm
- C) Number of lines of code
- D) Disk space used

Q3. [Def] Which of the following is a non-linear data structure?

- A) Array
- B) Linked List
- C) Stack
- D) Tree

Q4. [Def] Big O notation describes:

- A) The best-case scenario
- B) The exact running time
- C) The upper bound of complexity
- D) Average case only

Q5. [Def] The term 'Abstract Data Type' (ADT) refers to:

- A) Implementation details
- B) Logical description of data and operations
- C) Hardware architecture
- D) Operating system services

Q6. [Def] Which complexity is better for large input sizes?

- A) $O(n^2)$

- B) $O(n \log n)$
- C) $O(2^n)$
- D) $O(n!)$

Q7. [Output] What is the time complexity of a loop running from 1 to n with a step of $i = i * 2$?

- A) $O(n)$
- B) $O(\log n)$
- C) $O(n^2)$
- D) $O(1)$

Q8. [Def] A data structure where elements are accessed in $O(1)$ time using an index is:

- A) Linked List
- B) Stack
- C) Array
- D) Queue

Q9. [Def] What is 'Stable Sorting'?

- A) Uses minimal memory
- B) Maintains relative order of equal elements
- C) Always takes $O(n \log n)$
- D) Cannot be implemented recursively

Q10. [Def] Dynamic programming is mainly used for:

- A) Sorting
- B) Optimization problems with overlapping subproblems
- C) Linear search
- D) Memory management

Q11. [Def] Which of these is not an asymptotic notation?

- A) Omega
- B) Theta
- C) Delta
- D) Big O

Q12. [Error] Identify the error: Time complexity is always measured in seconds.

- A) Correct statement
- B) Incorrect, it measures growth relative to input
- C) Incorrect, it measures lines of code
- D) Incorrect, it measures file size

Q13. [Def] An algorithm is said to be efficient if:

- A) It uses more memory
- B) It has low time and space complexity
- C) It is written in C++
- D) It is very long

Q14. [Def] Best case complexity of Linear Search is:

- A) $O(n)$
- B) $O(\log n)$
- C) $O(1)$
- D) $O(n^2)$

Q15. [Def] A static data structure has:

- A) Variable size
- B) Fixed size allocated at compile time
- C) Size that grows at runtime
- D) No size limit

Arrays

Q16. [Error] What is the result of arr[5] in an array of size 5?

```
int arr[5] = {1, 2, 3, 4, 5};
```

- A) The last element
- B) 0
- C) Index Out of Bounds Error
- D) Null

Q17. [Def] Multi-dimensional arrays are stored in memory in:

- A) Random order
- B) Row-major or Column-major order
- C) Tree format
- D) Stack format

Q18. [Output] What is the output?

```
int a[] = {1,2,3,4}; System.out.print(a[2]);
```

- A) 1
- B) 2
- C) 3
- D) 4

Q19. [Def] Base address of an array is:

- A) Address of last element
- B) Address of first element
- C) Middle address
- D) Size of array

Q20. [Def] Time complexity to insert at the end of a dynamic array (average):

- A) $O(1)$
- B) $O(n)$
- C) $O(\log n)$
- D) $O(n^2)$

Q21. [Error] What happens here?

```
int arr[] = {1, 2, 3};  
for(int i=0; i<=3; i++) { System.out.print(arr[i]); }
```

- A) Prints 1 2 3
- B) Prints 1 3
- C) Array Index Out of Bounds
- D) Infinite Loop

Q22. [Def] Address of arr[i] is calculated as:

- A) Base + i
- B) Base + i * size_of_element
- C) Base - i
- D) i * size_of_element

Q23. [Def] Search complexity in a sorted array using Binary Search:

- A) $O(n)$
- B) $O(\log n)$
- C) $O(1)$
- D) $O(n \log n)$

Q24. [Output] Output?

```
int x[] = {1,2,3,4}; int s=0; for(int i:x) s+=i; System.out.print(s);
```

- A) 4
- B) 5
- C) 6
- D) 10

Q25. [Def] Arrays are passed to functions by:

- A) Value
- B) Reference/Pointer
- C) Copy
- D) Name

Q26. [Def] Sparse matrix has:

- A) Many non-zero elements
- B) Many zero elements
- C) No zero elements
- D) Only one row

Q27. [Def] Which operation is $O(1)$ in an array?

- A) Search
- B) Access by index
- C) Deletion from middle
- D) Insertion at start

Q28. [Error] Identify the error:

```
arr[-1] = 10;
```

- A) Array size must be constant
- B) Array elements must be same type
- C) Array index can be negative
- D) Arrays use contiguous memory

Q29. [Output] Result of `a[1][1]`?

```
int a[][] = {{1,2},{4,5}};
```

- A) 1
- B) 4
- C) 5
- D) 2

Q30. [Def] Deleting an element from the middle of an array requires:

- A) $O(1)$ time
- B) Shifting of elements
- C) Reversing the array
- D) No effort

Strings

Q31. [Def] In C, strings are terminated by:

- A) '\n'
- B) '\0'
- C) '.'
- D) ''

Q32. [Def] Strings in Java are:

- A) Mutable
- B) Immutable
- C) Arrays of integers
- D) Only 10 characters long

Q33. [Output] Output?

```
String s = "Hello"; System.out.print(s.charAt(1));
```

- A) H
- B) e
- C) l
- D) Error

Q34. [Def] What is a substring?

- A) A reversed string
- B) A contiguous part of a string
- C) A non-contiguous part
- D) A string length

Q35. [Error] Error in this code?

```
String s = "Java"; s[0] = 'j';
```

- A) Syntax error
- B) String is immutable
- C) Index out of bounds
- D) No error

Q36. [Def] Time complexity to find string length by traversing:

- A) $O(1)$
- B) $O(n)$
- C) $O(\log n)$
- D) $O(n^2)$

Q37. [Output] Output?

```
String a = "dsa"; System.out.print(a+a);
```

- A) dsadsa
- B) dsa
- C) Error
- D) dsa3

Q38. [Def] Lexicographical order is:

- A) Length based
- B) Dictionary order
- C) Random
- D) Frequency based

Q39. [Def] Function to concatenate two strings in C:

- A) strcpy()
- B) strcat()
- C) strlen()
- D) strcmp()

Q40. [Output] Output of length() for "Hi":

- A) 1
- B) 2
- C) 3
- D) 0

Q41. [Def] Which algorithm is used for pattern matching?

- A) Binary Search
- B) KMP Algorithm
- C) Bubble Sort
- D) Dijkstra

Q42. [Def] Anagram check requires:

- A) Same length only
- B) Same characters with same frequency
- C) Same starting letter
- D) Reversing one string

Q43. [Error] Identify error:

```
NullPointerException
```

- A) String s = "";
- B) s.length();
- C) s = null; s.charAt(0);

- D) s + " ";

Q44. [Def] A palindrome string:

- A) Reads same forward and backward
- B) Has no vowels
- C) Is uppercase
- D) Is empty only

Q45. [Def] Conversion of 'A' to 'a' involves:

- A) Adding 32 to ASCII
- B) Subtracting 32
- C) Reversing
- D) No change

Recursion

Q46. [Def] A base case is necessary to:

- A) Speed up code
- B) Stop infinite recursion
- C) Call another function
- D) Print the output

Q47. [Def] Recursion uses which internal data structure?

- A) Queue
- B) Stack
- C) Tree
- D) Linked List

Q48. [Output] Output of f(3)?

```
void f(int n) {
    if(n==0) return;
    System.out.print(n+" ");
    f(n-1);
}
```

- A) 3 2 1
- B) 1 2 3
- C) 3 3 3
- D) Error

Q49. [Error] Stack overflow occurs when:

- A) Base case is reached early
- B) Too many recursive calls without reaching base case
- C) The array is full
- D) Memory is cleared

Q50. [Def] Tail recursion is:

- A) Recursive call is at the start
- B) Recursive call is the last action
- C) No base case
- D) Multiple recursive calls

Q51. [Def] Fibonacci sequence is best solved by:

- A) Linear iteration
- B) Recursion (though inefficient without memoization)
- C) Sorting
- D) Queue

Q52. [Output] What is the output of fun(4)?

```
int fun(int n) {
    if(n==1) return 1;
    return n * fun(n-1);
}
```

- A) 4
- B) 10
- C) 24
- D) 1

Q53. [Def] Time complexity of Towers of Hanoi:

- A) $O(n)$
- B) $O(2^n)$
- C) $O(n^2)$
- D) $O(\log n)$

Q54. [Def] Direct recursion is when:

- A) A calls B, B calls A
- B) A calls itself
- C) A calls B
- D) No calls made

Q55. [Output] Output of fun(3)?

```
void fun(int n) {
    if(n==0) return;
    fun(n-1);
    System.out.print(n+" ");
}
```

- A) 1 2 3
- B) 3 2 1
- C) 2 1 3
- D) Error

Q56. [Error] Identify the error in this factorial function:

```
int fact(int n) { if(n==0) return 1; return n * fact(n-1); }
```

- A) Wrong math
- B) Infinite recursion for $n < 0$
- C) Missing parameter
- D) Syntax error

Q57. [Def] Indirect recursion involves:

- A) A single function
- B) A cycle of function calls
- C) No functions
- D) Iteration

Q58. [Def] Space complexity of recursion is due to:

- A) Array allocation
- B) Function call stack frames
- C) Global variables
- D) Heap memory

Q59. [Output] Result of sum(3)?

```
int sum(int n) {
    if(n==0) return 0;
    return n + sum(n-1);
}
```

- A) 3
- B) 6
- C) 5
- D) 0

Q60. [Def] Converting recursion to iteration usually requires:

- A) A tree

- B) An explicit stack
- C) A faster CPU
- D) More parameters

Singly Linked List

Q61. [Def] A node in a singly linked list contains:

- A) Data only
- B) Data and a pointer to next node
- C) Data and a pointer to previous node
- D) Two data fields

Q62. [Def] Time complexity to insert at the beginning:

- A) $O(1)$
- B) $O(n)$
- C) $O(\log n)$
- D) $O(n^2)$

Q63. [Def] To traverse a list, we start from:

- A) Tail
- B) Head
- C) Middle
- D) Any node

Q64. [Output] Output of temp.data?

```
Node head = new Node(10); head.next = new Node(20); Node temp = head.next;
```

- A) 10
- B) 20
- C) Null
- D) Error

Q65. [Def] Last node's next pointer points to:

- A) Head
- B) NULL
- C) Self
- D) Middle node

Q66. [Error] Identify the error: Removing the only node in the list.

- A) `head = head.next`

- B) head = null
- C) free(head)
- D) Both B and C depending on language

Q67. [Def] Searching for an element takes:

- A) $O(1)$
- B) $O(\log n)$
- C) $O(n)$
- D) $O(n \log n)$

Q68. [Def] Linked lists are better than arrays for:

- A) Random access
- B) Frequent insertions/deletions
- C) Binary search
- D) Memory locality

Q69. [Def] What is 'curr.next = curr.next.next' used for?

- A) Insertion
- B) Deletion of next node
- C) Reversing
- D) Traversal

Q70. [Output] What is the output?

```
head(10)->next(20). Traverse and print.
```

- A) 10 20
- B) 20 10
- C) 10
- D) 20

Q71. [Error] Identify error:

```
NullPointerException
```

- A) node.next = head;
- B) head = node;
- C) head.next.next = null;
- D) node = head.next; (when head is null)

Q72. [Def] A 'Head' pointer stores:

- A) First data value
- B) Address of the first node

- C) Number of nodes
- D) Address of the last node

Q73. [Def] Memory for nodes is allocated:

- A) At compile time
- B) Dynamically at runtime
- C) In the stack
- D) Contiguously

Q74. [Def] To insert after a given node 'prev':

- A) prev.next = newNode; newNode.next = prev.next;
- B) newNode.next = prev.next; prev.next = newNode;
- C) prev = newNode;
- D) head = newNode;

Q75. [Def] Self-referential structure means:

- A) Pointer to global variable
- B) Pointer to same structure type
- C) Pointer to itself only
- D) No pointers

Doubly & Circular Linked Lists

Q76. [Def] Doubly linked list nodes have:

- A) Two data fields
- B) One next pointer
- C) Next and Prev pointers
- D) No pointers

Q77. [Def] In a circular linked list, the last node points to:

- A) NULL
- B) Head
- C) Second node
- D) Previous node

Q78. [Def] Benefit of Doubly Linked List:

- A) Uses less memory
- B) Bidirectional traversal
- C) Faster than arrays
- D) Simpler implementation

Q79. [Error] Identify error in circular list traversal:

`Infinite loop if checking for null`

- A) temp != null
- B) temp != head (after first step)
- C) temp == head
- D) temp.next == null

Q80. [Def] To delete the only node in circular list:

- A) head = null
- B) head = head.next
- C) head.next = null
- D) tail = null

Q81. [Output] Output of curr.prev.data?

`10 <-> 20 <-> 30. curr is at 20.`

- A) 10
- B) 20
- C) 30
- D) Error

Q82. [Def] Space complexity of Doubly vs Singly:

- A) Same
- B) Doubly uses more (extra pointer)
- C) Singly uses more
- D) Depends on data

Q83. [Def] Which is used in browser back/forward?

- A) Singly
- B) Doubly
- C) Circular
- D) Stack

Q84. [Def] Time complexity to delete tail in Doubly (with tail pointer):

- A) $O(1)$
- B) $O(n)$
- C) $O(\log n)$
- D) $O(n^2)$

Q85. [Def] Josephus problem is solved using:

- A) Stack
- B) Circular Linked List
- C) Array
- D) Tree

Q86. [Error] Identify error:

Incorrect logic for DLL

- A) `p.next.prev = p;`
- B) `p.prev.next = p;`
- C) `p.next = p.prev;`
- D) `p.prev = null;` (at head)

Q87. [Def] Header circular list has:

- A) No head
- B) A special dummy node
- C) Two heads
- D) No tail

Q88. [Def] Accessing tail in Circular list (with tail pointer only) takes:

- A) $O(n)$
- B) $O(1)$
- C) $O(\log n)$
- D) $O(n^2)$

Q89. [Def] DLL insertion requires updating how many pointers?

- A) 1
- B) 2
- C) 4
- D) 0

Q90. [Def] Which is a valid Circular DLL node?

- A) next, prev, data
- B) next, data
- C) prev, data
- D) data only

Stacks

Q91. [Def] Stack follows the principle:

- A) FIFO
- B) LIFO
- C) Random
- D) Priority

Q92. [Def] Operation to add an element:

- A) Pop
- B) Push
- C) Enqueue
- D) Peek

Q93. [Def] Peek operation returns:

- A) Bottom element
- B) Top element without removing
- C) Size of stack
- D) Null

Q94. [Output] Output?

```
push(1); push(2); pop(); push(3); peek();
```

- A) 1
- B) 2
- C) 3
- D) Error

Q95. [Error] Stack underflow occurs during:

- A) Push
- B) Pop on empty stack
- C) Overflow
- D) Peek on full stack

Q96. [Def] Which is an application of Stack?

- A) Print queue
- B) Expression evaluation (Postfix)
- C) BFS
- D) Level order traversal

Q97. [Def] Time complexity of push/pop:

- A) $O(1)$
- B) $O(n)$
- C) $O(\log n)$

- D) $O(n^2)$

Q98. [Def] Recursive function calls are managed by:

- A) Heap
- B) System Stack
- C) Queue
- D) Array

Q99. [Def] Reverse a string using stack involves:

- A) Push all, Pop all
- B) Peek only
- C) Push half
- D) Circular stack

Q100. [Output] What is the output?

```
push(A); push(B); push(C); while(!empty) print(pop());
```

- A) A B C
- B) C B A
- C) B A C
- D) Error

Q101. [Error] Identify error in array implementation of Stack:

```
Overflow check
```

- A) `top++` before push
- B) `top = -1` initially
- C) `top++` after push when `top == size-1`
- D) `top--` on pop

Q102. [Def] Infix to Postfix conversion uses:

- A) Stack
- B) Queue
- C) Tree
- D) List

Q103. [Def] To implement Stack using Queue, how many queues are needed minimum?

- A) 1
- B) 2
- C) 3

- D) 4

Q104. [Def] A stack is full when:

- A) `top == size`
- B) `top == size - 1`
- C) `top == -1`
- D) `top == 0`

Q105. [Def] Balanced Parentheses problem uses:

- A) Queue
- B) Stack
- C) Array
- D) Recursion only

Queues

Q106. [Def] Queue follows the principle:

- A) LIFO
- B) FIFO
- C) Random
- D) Priority

Q107. [Def] Insertion in a queue is called:

- A) Push
- B) Pop
- C) Enqueue
- D) Dequeue

Q108. [Def] Dequeue removes from which end?

- A) Front
- B) Rear
- C) Top
- D) Middle

Q109. [Output] Output?

```
enqueue(1); enqueue(2); dequeue(); peek();
```

- A) 1
- B) 2
- C) 3

- D) Error

Q110. [Def] Queue overflow check in array implementation:

- A) rear == size - 1
- B) front == rear
- C) front == -1
- D) rear == 0

Q111. [Error] Identify error in Circular Queue full condition:

- A) rear == front
- B) (rear + 1) % size == front
- C) rear == size - 1
- D) front == 0

Q112. [Def] Application of Queue:

- A) Undo in editor
- B) Scheduling in OS (CPU)
- C) DFS
- D) Syntax parsing

Q113. [Def] A Deque is a:

- A) Stack
- B) Double Ended Queue
- C) Priority Queue
- D) Circular Queue

Q114. [Def] Time complexity of enqueue in a simple queue:

- A) $O(1)$
- B) $O(n)$
- C) $O(\log n)$
- D) $O(n^2)$

Q115. [Output] What is the output?

```
enqueue(1); enqueue(2); enqueue(3); while(!empty) print(dequeue());
```

- A) 1 2 3
- B) 3 2 1
- C) 1 3
- D) Error

Q116. [Error] Queue underflow condition:

- A) front == -1
- B) rear == -1
- C) front > rear
- D) Both A and C depending on implementation

Q117. [Def] Priority Queue elements are removed based on:

- A) Time of arrival
- B) Priority value
- C) Randomly
- D) Size

Q118. [Def] Circular Queue avoids:

- A) Memory wastage (false overflow)
- B) Underflow
- C) Complexity
- D) Use of pointers

Q119. [Def] To implement Queue using Stack, how many stacks are needed?

- A) 1
- B) 2
- C) 3
- D) 4

Q120. [Def] BFS (Breadth First Search) uses:

- A) Stack
- B) Queue
- C) List
- D) Array

Binary Trees

Q121. [Def] A tree with at most two children per node is:

- A) Binary Tree
- B) B-Tree
- C) AVL Tree
- D) Ternary Tree

Q122. [Def] Level of root node is usually:

- A) 1
- B) 0

- C) -1
- D) n

Q123. [Def] Inorder traversal order:

- A) Root-Left-Right
- B) Left-Root-Right
- C) Left-Right-Root
- D) Root-Right-Left

Q124. [Output] What is the Preorder of: 1 (root), 2 (left), 3 (right)?

- A) 2 1 3
- B) 1 2 3
- C) 2 3 1
- D) 3 2 1

Q125. [Def] Leaf nodes are those which have:

- A) Only one child
- B) No children
- C) Two children
- D) No parent

Q126. [Def] Total nodes in a full binary tree of height 'h' (starting from 0):

- A) 2^h
- B) $2^{(h+1)} - 1$
- C) $2^h - 1$
- D) h^2

Q127. [Output] Postorder traversal of: 1(root), 2(left), 3(right)?

- A) 2 3 1
- B) 1 2 3
- C) 3 2 1
- D) 2 1 3

Q128. [Error] Identify error: A tree cannot have cycles.

- A) Correct, it is an acyclic graph
- B) Incorrect, it must have cycles
- C) Incorrect, only binary trees are acyclic
- D) Correct, but only for directed trees

Q129. [Def] Height of a tree is:

- A) Number of nodes
- B) Max depth from root to leaf
- C) Number of edges in longest path
- D) Both B and C depending on definition

Q130. [Def] Which traversal is used for Level Order?

- A) DFS
- B) BFS
- C) Inorder
- D) Postorder

Q131. [Def] Full Binary Tree means:

- A) Every node has 0 or 2 children
- B) Every node has 2 children
- C) All levels are filled
- D) Tree is balanced

Q132. [Error] Identify error in recursive Inorder:

- A) Missing left call
- B) Missing root print
- C) Missing base case 'if(root == null) return'
- D) All of above

Q133. [Def] Degree of a node is:

- A) Number of children
- B) Level number
- C) Total nodes in subtree
- D) Distance from root

Q134. [Def] Which traversal gives non-decreasing order for BST?

- A) Preorder
- B) Inorder
- C) Postorder
- D) Level order

Q135. [Def] Height of a single node tree is:

- A) 0
- B) 1
- C) NULL
- D) 2

BST

Q136. [Def] In a BST, the left child is always:

- A) Greater than root
- B) Smaller than root
- C) Equal to root
- D) Greater than right child

Q137. [Def] In a BST, the right child is always:

- A) Smaller than root
- B) Greater than root
- C) NULL
- D) Even

Q138. [Def] Time complexity for search in a balanced BST:

- A) $O(n)$
- B) $O(\log n)$
- C) $O(1)$
- D) $O(n \log n)$

Q139. [Def] What happens to the Inorder traversal of a BST?

- A) It is random
- B) It is in ascending order
- C) It is in descending order
- D) It repeats root

Q140. [Error] Identify error: Inserting 5 into a BST where root is 10 and 5 already exists in left subtree.

- A) 5 goes to right of 10
- B) 5 is ignored or handled as duplicate
- C) BST becomes a graph
- D) Root becomes 5

Q141. [Def] To find the minimum element in a BST, we move:

- A) Right as far as possible
- B) Left as far as possible
- C) To the root
- D) To any leaf

Q142. [Def] Successor of a node in Inorder traversal is:

- A) Smallest node in right subtree
- B) Largest node in left subtree
- C) Parent node
- D) Left child

Q143. [Def] Complexity of insertion in worst-case (Skewed BST):

- A) $O(\log n)$
- B) $O(n)$
- C) $O(1)$
- D) $O(n^2)$

Q144. [Output] Output of searching for 15 in BST with root 10?

- A) Look in left
- B) Look in right
- C) Not found
- D) Error

Q145. [Def] Deleting a node with two children in BST involves replacing it with:

- A) Its parent
- B) Its Inorder Successor or Predecessor
- C) NULL
- D) Its sibling

Q146. [Error] Identify error: BST root is 5, left is 10.

- A) Correct
- B) Incorrect, $10 > 5$
- C) Incorrect, root must be 0
- D) Correct if 10 is balanced

Q147. [Def] Range search in BST is efficient because of:

- A) Hashing
- B) Pruning subtrees using properties
- C) Contiguous memory
- D) Parallel processing

Q148. [Def] BST property must hold for:

- A) Only root
- B) Every node in the tree
- C) Only leaf nodes
- D) Only left nodes

Q149. [Def] Time complexity to find maximum element in balanced BST:

- A) $O(n)$
- B) $O(1)$
- C) $O(\log n)$
- D) $O(n \log n)$

Q150. [Def] An AVL tree is a self-balancing:

- A) Stack
- B) Heap
- C) Binary Search Tree
- D) Linked List

Section 2: Code & Debugging

1. What is the error in this snippet?

```
int arr[] = {1, 2, 3, 4, 5};
for(int i = 0; i <= arr.length; i++){
    System.out.print(arr[i] + " ");
}
```

- A) Infinite loop
- **B) Index out of bounds due to <=**
- C) Skips last element
- D) No error

2. What is the issue with this recursion?

```
int sum(int n){
    if(n == 0) return 0;
    sum(n-1);
}
```

- A) Missing base case
- **B) Missing return statement in recursive call**
- C) Infinite recursion
- D) Wrong function name

3. Incorrect in this stack overflow check?

```
if(top == size){
    System.out.println("Overflow");
}
```

- **A) Should be top == size - 1**
- B) Should be top == 0

- C) Stack never overflows
- D) Condition is correct

4. **Wrong with this dequeue?**

```
int dequeue(){
    return arr[rear--];
}
```

- **A) Dequeue should happen from front**
- B) rear should increment
- C) Syntax error
- D) Works fine

5. **BST validation issue?**

```
if(root.left.data > root.data){
    return false;
}
```

- A) Correct BST validation
- **B) Only checks immediate child, not full subtree**
- C) Should use >=
- D) Syntax error

6. **Bug in this traversal?**

```
Node temp = head;
while(temp.next != null){
    System.out.print(temp.data + " ");
    temp = temp.next;
}
```

- A) Infinite loop
- B) First node skipped
- **C) Last node not printed**
- D) Syntax error

7. **Missing in maxDepth?**

```
int maxDepth(TreeNode root){
    if(root == null) return 0;
    return Math.max(maxDepth(root.left), maxDepth(root.right));
}
```

- A) Base case
- B) Recursive call
- **C) +1 for current node**
- D) Null check

8. **Wrong with circular queue full condition?**

```
if(rear == size - 1){  
    System.out.println("Full");  
}
```

- A) Condition is correct
- **B) Should use (rear + 1) % size == front**
- C) rear should be 0
- D) front should be incremented

9. **Issue with this DFS?**

```
void dfs(int node){  
    for(int neigh : adj[node]){  
        dfs(neigh);  
    }  
}
```

- A) Syntax error
- B) Missing adjacency list
- **C) No visited check -> infinite recursion**
- D) DFS cannot be recursive

10. **Bug in comparison?**

```
if(str1 == str2){  
    System.out.println("Equal");  
}
```

- A) Syntax error
- **B) Comparing references instead of values**
- C) Strings cannot be compared
- D) Should use compareTo() only

11. **What does q.peek() return?**

```
Queue<Integer> q = new LinkedList<>();  
q.add(1); q.add(2); q.remove(); q.add(3);
```

- A) 1
- **B) 2**
- C) 3
- D) Error

12. **Height of Tree: root(1), left(2), right(3)?**

- A) 1
- **B) 2**
- C) 3

- D) 0

13. Nodes in full binary tree with height = 2?

- A) 2
- **B) 3**
- C) 4
- D) 1

14. BST elements: 2, 1, 3. Which traversal prints 1 2 3?

- A) Preorder
- B) Postorder
- **C) Inorder**
- D) Level order

15. Output of Queue loop?

```
Queue<Integer> q = new LinkedList<>(); q.add(1); q.add(2); q.add(3);
while(!q.isEmpty()){ System.out.print(q.remove()+" "); }
```

- A) 3 2 1
- **B) 1 2 3**
- C) 2 1 3
- D) Infinite

16. Output of Stack loop?

```
Stack<Integer> st = new Stack<>(); st.push(1); st.push(2); st.push(3);
while(!st.isEmpty()){ System.out.print(st.pop()+" "); }
```

- A) 1 2 3
- **B) 3 2 1**
- C) 2 3 1
- D) Infinite

17. State of arr?

```
int arr[] = {4,3,2};
for(int i=0;i<2;i++){
    if(arr[i] > arr[i+1]){ int t = arr[i]; arr[i] = arr[i+1]; arr[i+1] = t;
    }
}
```

- A) 2 3 4
- **B) 3 2 4**
- C) 3 4 2
- D) 4 3 2

18. State of selection logic?

```
int arr[] = {3,1,2}; int min=0;
for(int j=1;j<3;j++){ if(arr[j] < arr[min]) min=j; }
int t = arr[0]; arr[0]=arr[min]; arr[min]=t;
```

- **A) 1 3 2**
- B) 1 2 3
- C) 3 1 2
- D) 2 1 3

19. What is the sum?

```
int arr[] = {1,2,3,4}; int i=0, j=3, sum=0;
while(i<=j){ sum += arr[i] + arr[j]; i++; j--; }
```

- **A) 10**
- B) 12
- C) 20
- D) 8

20. How many calls for fun(5)?

```
int fun(int n){ if(n==1) return 1; return fun(n-1); }
```

- A) 4
- **B) 5**
- C) 1
- D) Infinite

21. Final sum?

```
Stack<Integer> st = new Stack<>(); st.push(1); st.push(2); st.push(3);
int sum = 0; while(!st.isEmpty()){ sum += st.pop(); }
```

- **A) 6**
- B) 5
- C) 3
- D) 0

22. In a BST, the left child is:

- A. Greater
- **B. Smaller**
- C. Equal
- D. Random

23. Error in arr[5] = 10 for int arr[5]?

- A. No error
- **B. Index out of bounds**
- C. Syntax error
- D. Runtime exception

24. Error in `s[0] = 'h'` for `String s = 'Hello'`?

- A. No error
- **B. String is immutable**
- C. Syntax error
- D. Logical error

25. Problem in `void fun() { fun(); }`?

- A. No issue
- **B. Infinite recursion**
- C. Syntax error
- D. Compile error

26. Error on empty Stack pop?

- A. No error
- B. Stack overflow
- **C. Stack underflow**
- D. Syntax error

27. Error on empty Queue dequeue?

- A. Overflow
- **B. Underflow**
- C. Syntax error
- D. No error

28. Error on `null head.next`?

- A. Syntax error
- **B. Null pointer exception**
- C. Logical error
- D. No error

29. Error in `if(root.left > root)`?

- A. Syntax error
- **B. Comparing node incorrectly**
- C. No error
- D. Runtime error

30. Error in `int arr[] = new int[-5]`?

- A. No error
- **B. Negative size array**
- C. Syntax error
- D. Logical error

31. Issue if stack full on push?

- A. Underflow
- **B. Overflow**
- C. No issue
- D. Syntax error

32. Error in `node.prev.next = node` where `prev` is null?

- A. No error
- **B. Null pointer**
- C. Syntax error
- D. Logical error

33. Output of `arr[3]`?

```
int arr[] = {1,2,3,4};
for(int i=1;i<arr.length;i++){
    arr[i] = arr[i] + arr[i-1];
}
```

- A) 4
- B) 6
- **C) 10**
- D) 7

34. Output of `arr[2]`?

```
int arr[] = {1,2,3,4,5};
int i=0, j=4;
while(i<j){
    int temp = arr[i]; arr[i] = arr[j]; arr[j] = temp;
    i++; j--;
}
```

- A) 1
- **B) 3**
- C) 5
- D) 4

35. Output of `count`?

```
String s = "aab"; int count = 0;
for(int i=0;i<s.length();i++){
    if(s.charAt(i)=='a') count++;
}
```

- A) 1
- **B) 2**
- C) 3
- D) 0

36. Output of fun(4)?

```
int fun(int n){
    if(n==0) return 0;
    return n + fun(n-1);
}
```

- A) 6
- **B) 10**
- C) 8
- D) 4

37. Output of fun(3)?

```
void fun(int n){
    if(n==0) return;
    fun(n-1);
    System.out.print(n+" ");
}
```

- A) 3 2 1
- **B) 1 2 3**
- C) 2 1 3
- D) 1 3 2

38. Output of sum?

```
int arr[] = {1,2,3,4}; int i=0, sum=0;
while(i<3){ sum += arr[i]; i++; }
```

- **A) 6**
- B) 10
- C) 7
- D) 9

39. What does count represent?

```
Node temp = head;
while(temp != null){ count++; temp = temp.next; }
```


- **A) Number of nodes**
- B) Always 0
- C) Last node value
- D) Error

40. **Output of prev.data after reversal?**

```
Node prev = null, curr = head;
while(curr != null){
    Node next = curr.next; curr.next = prev;
    prev = curr; curr = next;
}
```

- A) 1
- B) 2
- **C) 3**
- D) Error

41. **What does st.peek() return?**

```
Stack<Integer> st = new Stack<>();
st.push(1); st.push(2); st.pop(); st.push(3);
```

- A) 1
- B) 2
- **C) 3**
- D) Error

